

PAAF

Polymer Auto-Assembly Framework

Complete User Manual & Methods Guide

PAAF is a desktop application that turns a single polymer monomer into a fully-typed, packed molecular-dynamics (MD) system for LAMMPS or GROMACS. It automates every step between a chemical structure and a ready-to-run simulation: chain building, geometry optimisation, force-field assignment, periodic-box packing, crosslinking reactions, and multi-component blends. This guide explains every page and every option, then documents the scientific methods behind them.

Version	0.1.0
Launch	<code>python run_paaf.py</code> (from the PAAF folder, conda env active)
Interface	PyQt5 desktop GUI — dark sidebar, page in the centre, log console at the bottom
Pipeline (6 steps)	Builder → Chain → Optimize → Force field → Box → Export
Independent tools	Reactions · System builder · Blend
MD engines	LAMMPS and GROMACS
Force fields	22 (OPLS-AA, OPLS 2005, PCFF, COMPASS, CVFF, GAFF, CHARMM36, GROMOS 54A7, TraPPE, DREIDING, MARTINI, ...)
Polymer library	124 curated + database polymers with Tg and density

1 How PAAF Works — the big picture

Part I · User Manual

2 Builder — define your monomer (Step 1)

3 Chain — polymerise into a chain (Step 2)

4 Optimize — relax the chain (Step 3)

5 Force field — parameters & MD engine (Step 4)

6 Box — pack the periodic cell (Step 5)

7 Export — generate topology files (Step 6)

8 Reactions — crosslinking

9 System builder — cells, surfaces, layers

10 Blend — multi-component packing

Part II · Methods & Architecture

A–G Loading · Chain assembly · Optimisation · Typing · Packing · Blends · Stack

1 How PAAF Works — the big picture

You give PAAF a monomer — as a structure file, a SMILES string, or a pick from the library — and it walks through six pipeline steps, each a page in the sidebar, producing a complete MD system at the end. The sidebar's top six entries are the linear pipeline (with Back / Next buttons); the bottom three are independent tools that operate on systems the pipeline produced.

The pipeline, end to end:

- **Builder** — define the monomer(s). Upload a file, compose a SMILES, or pick from the 124-polymer library. Everything lands in a shared Monomers table used by every later step.
- **Chain** — polymerise the monomer into N repeat units. Choose homopolymer or copolymer, the sequence, and how many units.
- **Optimize** — relax the assembled chain's geometry with an OpenBabel force field so bond lengths are physical before typing.
- **Force field** — choose the MD force field; PAAF assigns atom types. For DL_FIELD force fields you also pick the MD output engine here.
- **Box** — pack N chains into a periodic cell of a chosen shape, size, or target density.
- **Export** — write the final LAMMPS or GROMACS topology files.

The three independent tools:

- **Reactions** — learn crosslink/reaction templates from reactant→product pairs and apply them to a packed system.
- **System builder** — amorphous cells, crystals, surfaces, nanotubes, solvated boxes, and multilayer stacks.
- **Blend** — pack two or more different polymers into one box with merged topology.

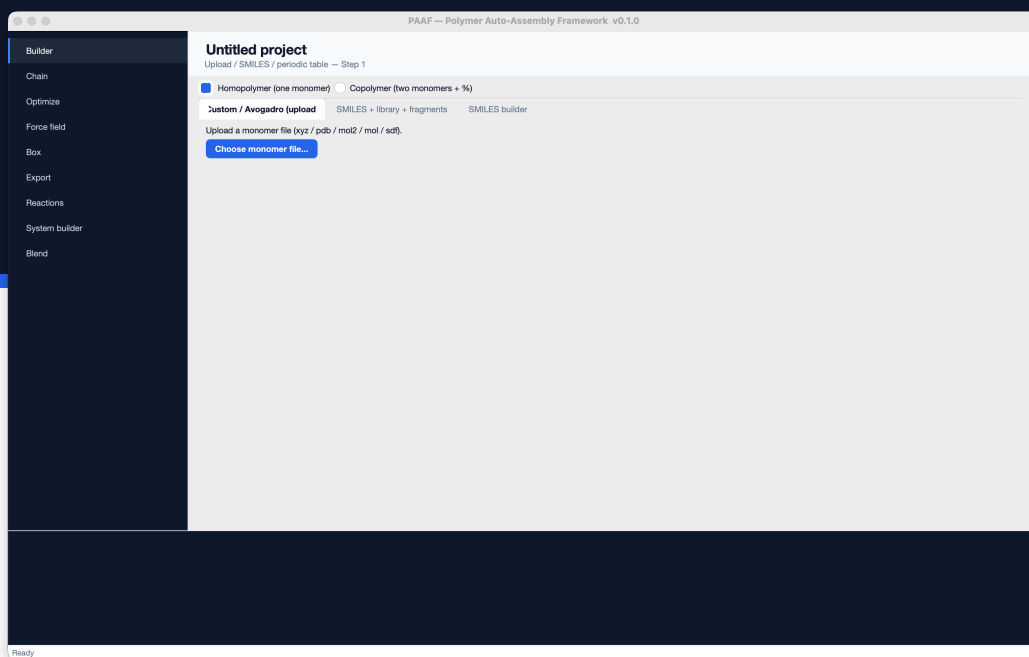


Figure 1 — The PAAF main window: dark sidebar navigation (left), the active page (centre), and the log console (bottom).

2 Builder — define your monomer (Step 1)

tabs, and at the bottom sits the shared **Monomers** table.

The three input tabs

- **Custom / Avogadro (upload)** — load a monomer geometry you built in another program (XYZ, PDB, MOL2, MOL, SDF). Use this for structures from Avogadro, ChemDraw, Materials Studio, etc. Tip: cap both ends with hydrogens at the connection points, and export with bonds (MOL2 / PDB-with-CONECT) so PAAF reads the connectivity correctly.
- **SMILES + library + fragments** — search the 124-polymer library (PBS, PET, PE, PS, nylons, natural rubber, polychloroprene, ...), each row showing its glass-transition temperature T_g , density, and polymerization SMILES. Or type a SMILES directly, or append pre-defined fragments. Picking a library polymer records its `[*]...[*]` polymerization SMILES so the Chain step knows exactly which atoms connect.
- **SMILES builder** — a periodic table plus a SMILES-notation strip (bonds, branches, rings, stereo, brackets, charges, wildcards) for composing a SMILES click by click without memorising the syntax.

The shared Monomers table

Every monomer you build appears here, and the Chain, Optimize, Force-field, Box and Export pages all read from it. For uploaded files you can double-click the *Head / Tail / Head_H / Tail_H* columns to specify the exact connection atoms as 1-based indices — this bypasses PAAF's automatic guess and guarantees the polymer bonds where you intend.

	PID	Name	T_g (K)	ρ (kg/m ³)	SMILES	src
1	W01_P103	Poly(1-heptene)	228.1526	835.0498	<chem>[*]CC(CCCCC[*])</chem>	DB
2	W01_P102	Poly(vinyl hexyl ether)	236.2172	894.7144	<chem>[*]CC(CCCCCO[*])</chem>	DB
3	W01_P101	Poly(methylglycidyl ether)	289.6251	1082.5284	<chem>[*]OCC(COC[*])</chem>	DB
4	W01_P100	Poly(hexane oxide)	255.5598	912.7678	<chem>[*]C(CCCO[*])</chem>	DB
5	W01_P099	Poly(1-hexene)	235.7869	833.8054	<chem>[*]C(CCCO[*])</chem>	DB
6	W01_P098	Poly(butylene succinate)	246.4947	1203.178	<chem>[*]OCCCCO(-O(CCC=O)O) DB</chem>	DB
7	W01_P097	Polychloroprene	277.527	1213.257	<chem>[*]C(C)=CC[*]</chem>	DB
8	W01_P096	Poly(4,4-dimethyl-1-pentene)	377.2094	822.1596	<chem>[*]C(C(C)C)C(C)C[*]</chem>	DB
9	W01_P095	nylon 12	252.2704	960.778	<chem>[*]C(-O)CCCCCCCCC(=O)N DB</chem>	DB
10	W01_P094	nylon 46	296.6291	1108.917	<chem>[*]C(-O)CCCCC(=O)NCCCCN[*] DB</chem>	DB

Figure 2 — Builder, SMILES + library tab: the searchable 124-polymer library with T_g , density and polymerization SMILES.

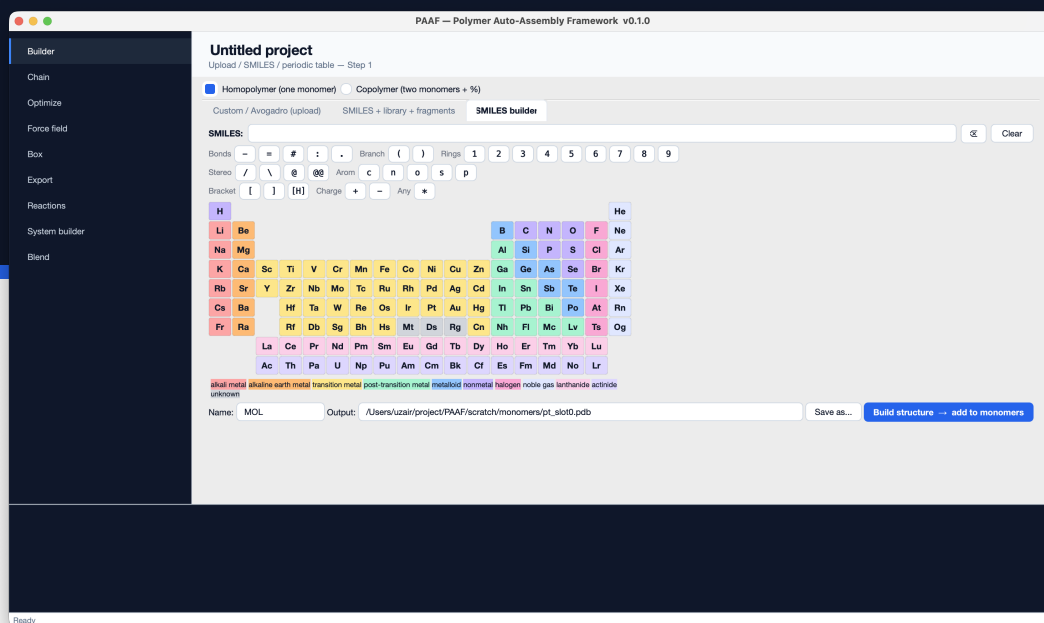


Figure 2b — SMILES builder tab: compose a SMILES from the notation strip and periodic table.

3 Chain — polymerise into a chain (Step 2)

The Chain page assembles the monomer into an N-unit polymer. For a homopolymer it simply repeats one monomer; for a copolymer it interleaves two according to the sequence mode and composition you set. The Copolymer helper lets you pick two monomers from the Builder table, set the fraction of B, and it fills the fields above automatically.

Total monomers per chain	Degree of polymerisation N (e.g. 20).
Sequence mode	homopolymer · alternating · block · random. The last three combine two monomers.
Backend	auto (recommended) · mbuild · simple. 'auto' uses the robust mBuild builder when its dependencies are installed.
Fractions (random)	Composition for random copolymers, e.g. 0.7,0.3.
Block sizes (block)	Repeat-unit run lengths for block copolymers, e.g. 5,5.
Random seed	Optional — fixes the random copolymer sequence for reproducibility.
Auto-cap polyester end	For monomers whose tail is a carbonyl carbon (PBS/PET/PLA), the chain end is capped as -C(=O)OH. No effect on polyethylene, polystyrene, etc.

For library monomers that carry a polymerization SMILES, PAAF uses a graph-isomorphism chain builder: it builds a clean 3D compound directly from the SMILES, locates the two connection atoms, and joins repeat units through them — giving straight, correctly-bonded backbones instead of the geometry artefacts a naïve builder can introduce.

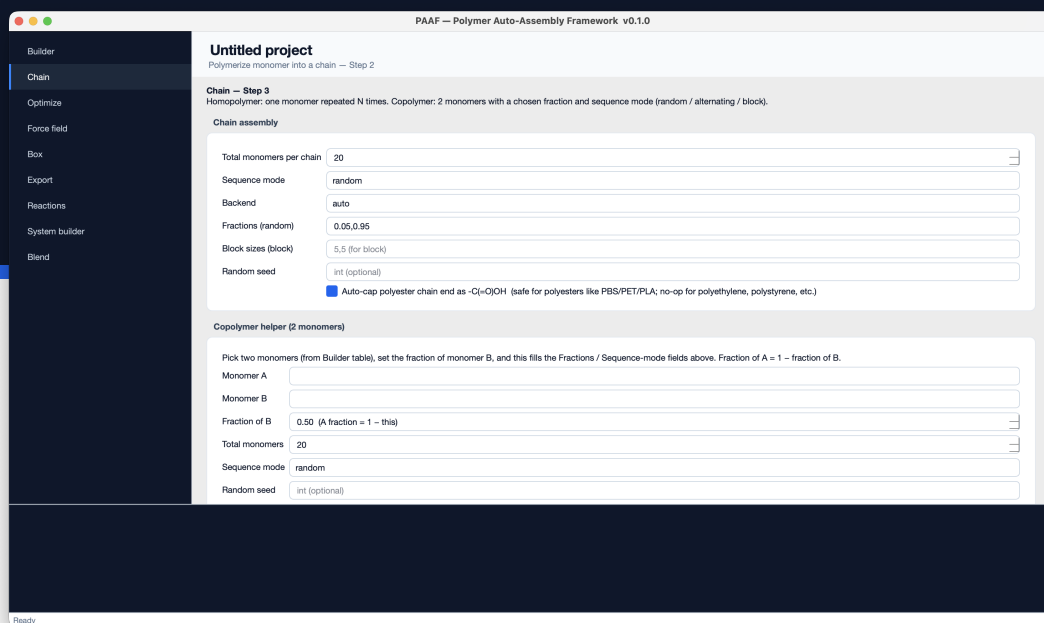


Figure 3 — Chain assembly settings, with the Copolymer helper below.

4 Optimize — relax the chain (Step 3)

Once assembled, the full chain is geometry-minimised with an OpenBabel force field so bond lengths and angles are physically sensible before atom typing. This step matters: DL_FIELD and moltemplate perceive bonds by distance, so a stretched or clashing chain could be mis-typed. You can turn optimisation off, but it is recommended for any chain longer than a few units.

Optimize monomers + chain	Master on/off switch for this step.
Force field	MMFF94 · MMFF94s · UFF · Ghemical · GAFF. If the chosen FF can't set up a molecule, PAAF falls back MMFF94 → UFF → Ghemical.
Max steps	Conjugate-gradient iteration cap (e.g. 10,000). Larger = tighter but slower; long polyester chains benefit from more steps.
Convergence tol	Energy tolerance in scientific notation, e.g. 1.0e-6.
Algorithm	Conjugate gradients (recommended) or steepest descent.
Log initial + final energy	Prints the energy before/after so you can see how much the geometry relaxed.

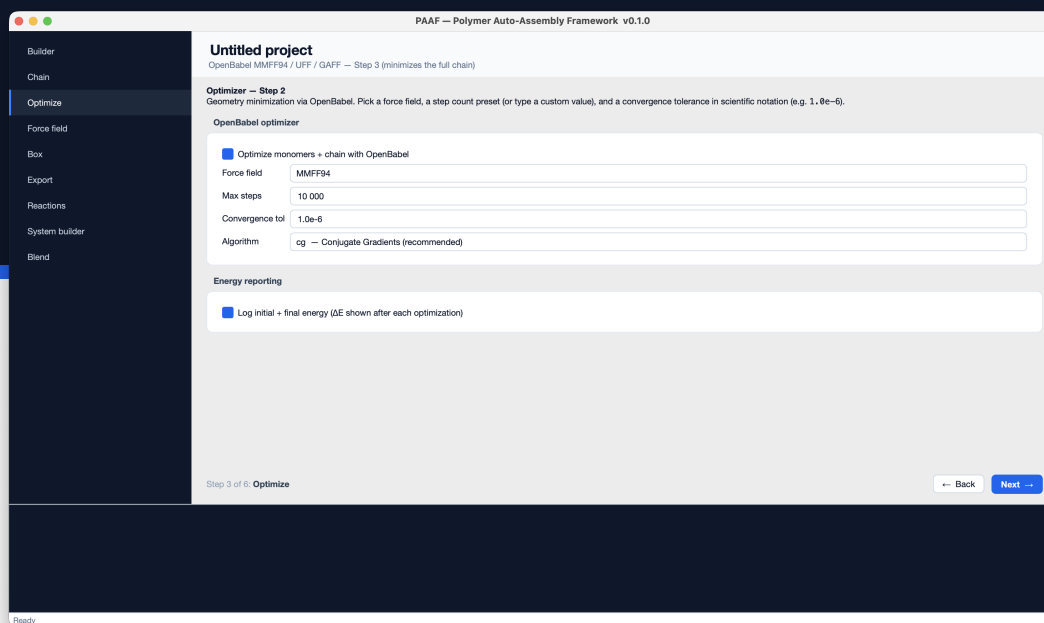


Figure 4 — OpenBabel optimisation controls.

5 Force field — parameters & MD engine (Step 4)

Here you pick the force field that parameterises the system. PAAF ships 22, in two families that differ in how atoms are typed and files are written:

- **Moltemplate-native** (OPLS-AA 2024/2008, L-OPLS-AA, TraPPE-UA, OPLS-UA, GAFF/GAFF2, DREIDING, COMPASS-published, SDK, MARTINI, Dry-MARTINI) — written as `.lt` files and compiled by `moltemplate.sh` into LAMMPS data/settings.
- **DL_FIELD-based** (OPLS 2005, PCFF, COMPASS, CVFF, CHARMM36, GROMOS 54A7, AMBER GAFF, TraPPE-EH, OPLS-UA) — typed by the `dl_field` program, which auto-assigns atom types against its library and can emit LAMMPS or GROMACS directly.

The default is **OPLS 2005 (DL_FIELD)**. When a DL_FIELD force field is selected, three things appear: an **MD output format** chooser (`lammps` / `gromacs` / `both`) that is written straight into the control file so `dl_field` produces the right topology; a **DL_FIELD lib dir** field pointing to your `dl_f_4.13/lib` directory (remembered once set and locked so it isn't changed by accident); and an editable **polymer.control** preview you can tweak before generation.

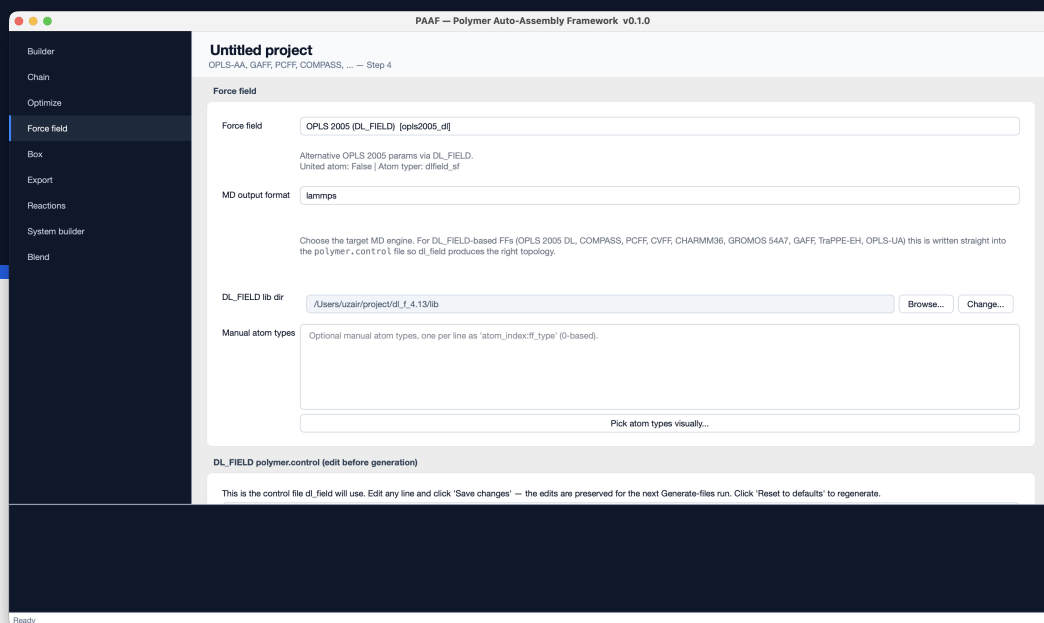


Figure 5 — Force-field page with OPLS 2005 (DL_FIELD) selected, the MD-output-format chooser, and the DL_FIELD library path.

OPLS-AA 2024 (Moltemplate native) [oplsaa]
OPLS-AA 2008 (Jorgensen) [oplsaa2008]
L-OPLS-AA 2024 (long-chain refit) [loplsaa]
L-OPLS-AA 2008 [loplsaa2008]
TraPPE-UA (Moltemplate native) [trappe_ua]
OPLS-UA 2024 (extracted from OPLS-AA 2024) [oplsua_2024]
GAFF (Moltemplate native, antechamber for typing) [gaff]
GAFF2 (Moltemplate native, antechamber) [gaff2]
DREIDING (Moltemplate native) [dreiding]
COMPASS (public parameters, Moltemplate native) [compass_published]
SDK coarse-grained lipids/cholesterol [sdk]
MARTINI 2.0 (coarse-grained) [martini]
Dry MARTINI (implicit solvent) [drymartini]
PCFF (converted from DL_FIELD) [pcff]
COMPASS (converted from DL_FIELD) [compass]
CVFF (converted from DL_FIELD) [cvff]
OPLS 2005 (DL_FIELD) [opls2005_dj]
OPLS-UA (DL_FIELD) [opls_ua]
TraPPE-EH (DL_FIELD) [trappe_eh]
CHARMM36 (DL_FIELD) [charmm36]
AMBER GAFF (DL_FIELD) [amber_gaff]
GROMOS 54A7 (DL_FIELD) [gromos_54a7]

Figure 5b — The 22 built-in force fields, spanning Moltemplate-native and DL_FIELD families.

Manual atom typing (optional)

For fine control, **Pick atom types visually...** opens a manual atom-typing dialog. The left panel lists every atom in your monomer with its detected chemical group and neighbours; the right panel is the searchable force-field atom-type library (id, element, key, partial charge, description). Click an atom, click a type, and assign — or press *Auto-type all* to seed everything from the SMARTS typer and then fine-tune. Assignments can be saved and reloaded as profiles for reuse across projects.

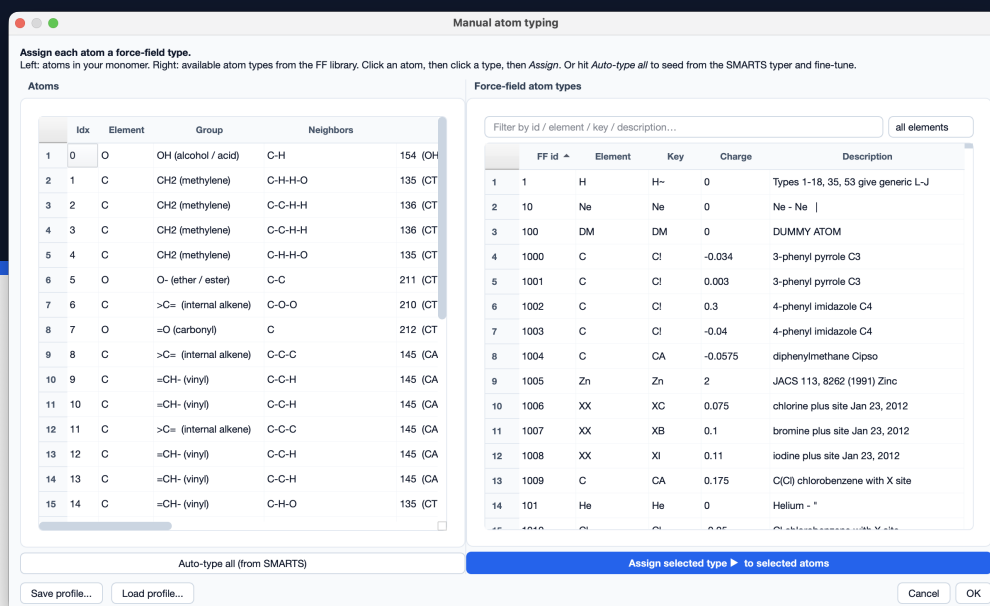


Figure 5c — Manual atom-typing dialog: per-atom force-field type assignment against a searchable type library.

6 Box — pack the periodic cell (Step 5)

The Box page packs multiple chains into a periodic simulation cell. You set how many chains, the cell shape and size (or a target density that derives the edge lengths), and the packing engine. For LAMMPS, packing uses `packmol`; for GROMACS it uses `gmx insert-molecules`.

Number of chains	How many chain copies to pack into the box.
Box shape	cubic · orthorhombic · triclinic. Cubic mirrors edge a into b and c.
Cell edge lengths	a, b, c in Ångström.
Target density	Optionally derive edge lengths from a density in g/cm ³ , preserving the chosen shape's aspect ratio.
Packmol binary	Path to packmol (auto-detected; editable).
Packmol seed	-1 = a fresh random layout every run; any positive integer fixes the layout for exact reproducibility.
Packmol tolerance	Minimum inter-molecule atom distance in Å (2.0 is safe; 1.5 packs tighter).
GROMACS packing	When the engine is GROMACS: choose <i>Explicit chain count</i> or <i>Target atom count</i> (inserts <code>atom_limit ÷ atoms-per-chain copies</code>), with optional pre-minimisation and a try-count. If gmx is not installed, PAAF falls back to a built-in pure-Python packer.

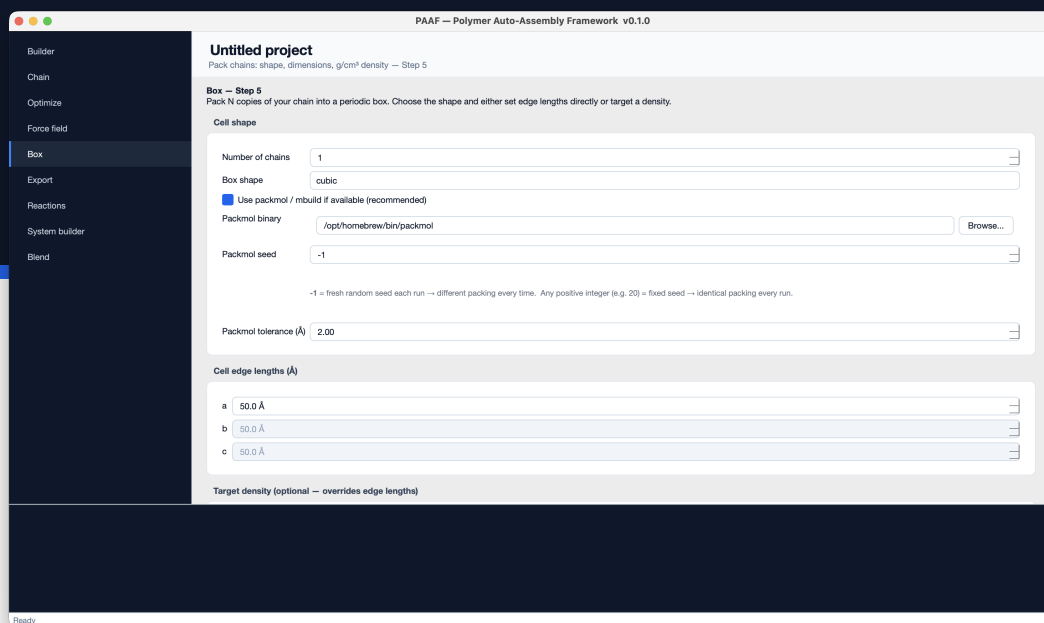


Figure 6 — Box page: cell shape, packmol / gmx options, dimensions, and density.

7 Export — generate topology files (Step 6)

The final step writes the MD files. Set the project name and output directory, confirm the **MD engine output** (lammps / gromacs / both) and the **topology builder** (auto / moltemplate.sh / dl_field), optionally give a GROMACS `#include .itp` path, then click **Generate files**. MD execution is off by default — PAAF writes topology, not trajectories.

What gets produced:

- **Moltemplate-native force fields** → `<project>.lt`, `system.lt`, `system.data`, `system.in.settings`, `system.in.init` via `moltemplate.sh`.
- **DL_FIELD force fields** → LAMMPS `lammps.data` + `lammps.in`, or GROMACS `system.gro` + `system.top` + `*.mdp`, via `dl_field`.
- Multi-chain packed boxes: `packed_box.data` (LAMMPS) or `packed_box.gro` (GROMACS).

PAAF deliberately writes no ensemble / thermostat / damping settings — set those in your own LAMMPS input or MDP when you run the simulation. You can also Save / Load the entire configuration so a run is exactly reproducible.

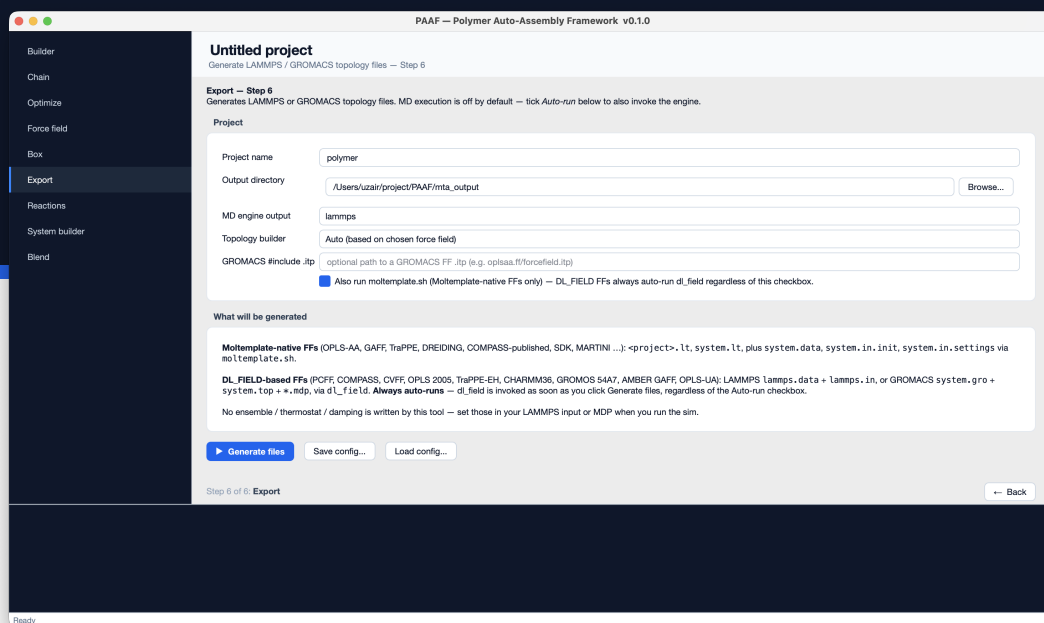


Figure 7 — Export page and the generated-file summary.

8 Reactions — crosslinking (independent tool)

The Reactions tool learns reaction templates from example structures. Add one row per reaction and upload its **reactant** and **product** structures; PAAF compares them by graph mapping to derive which bonds form or break, and stores the result as a reusable reaction library (JSON). Optionally it applies the learned templates to a packed system to build a crosslinked network — useful for thermosets, epoxies and vulcanised rubbers. A reactive-pair cutoff and a maximum-events cap control how aggressively crosslinks are formed.

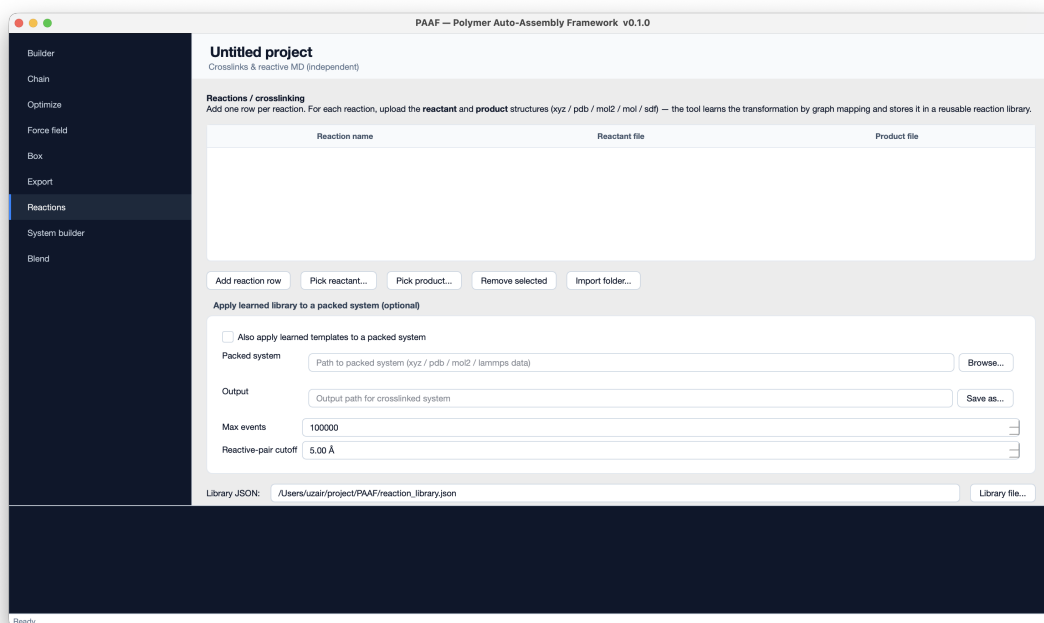


Figure 8 — Reactions / crosslinking tool.

9 System builder — cells, surfaces, layers (independent tool)

A Materials-Studio-style construction toolkit for starting configurations beyond a single packed polymer box. Pick a mode, add the species / parameters, and Build:

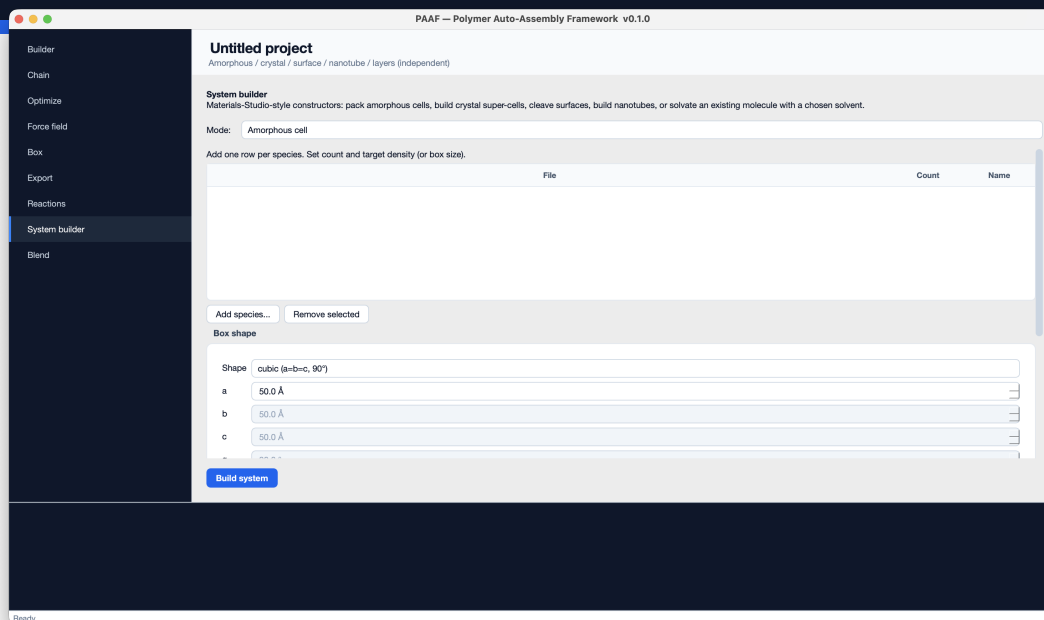


Figure 9 — System builder, Amorphous-cell mode.

10 Blend — multi-component packing (independent tool)

The Blend tool packs two or more *different* polymers into one box. Each component is a pre-built single-chain LAMMPS data file (typically the `dlf_output1/lammps1.data` a previous PAAF run produced) plus a copy count. PAAF offsets every atom / bond / angle / dihedral / improper type ID per component so they never collide, merges all the coefficient sections, packs every copy of every species with packmol, and writes a single consolidated `packed_blend.data` plus a matching `packed_blend.in` with offset `pair_coeff` commands. The *Import from PAAF run...* button auto-populates components from an existing output directory.

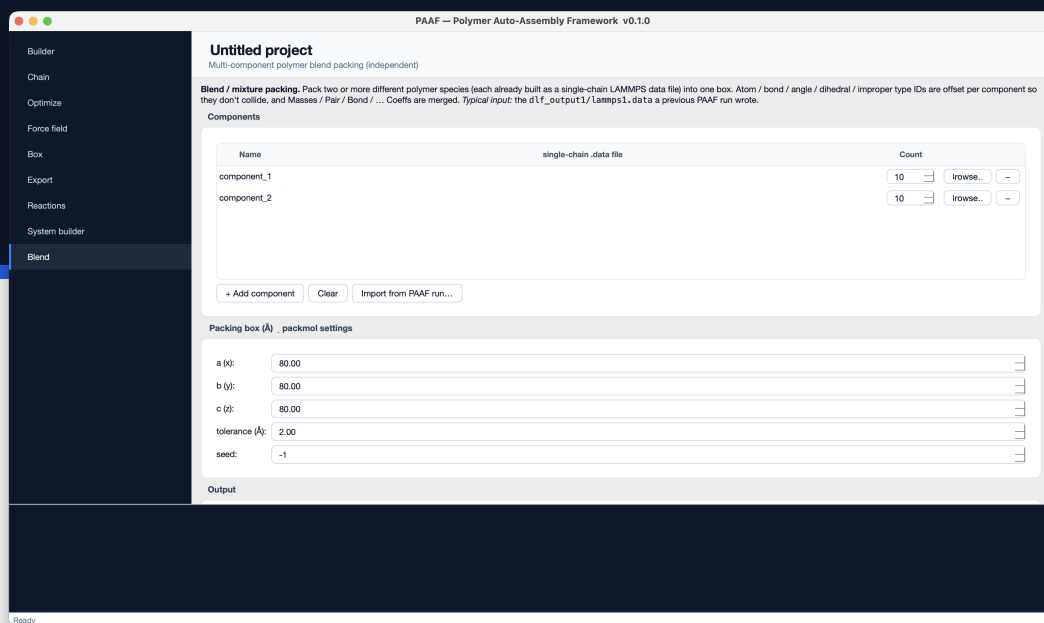


Figure 10 — Blend / mixture packing: components and packmol settings.

Identify the head and tail connection atoms by element, avoiding fragile geometry-based guessing. Uploaded files without wildcards use an antiparallel-vector heuristic, or the head/tail indices you enter in the Monomers table.

B. Chain assembly (mBuild + graph isomorphism)

The preferred builder constructs a clean 3D compound directly from SMILES via `mbuild.load(smiles=True)`, then uses RDKit + NetworkX graph isomorphism to map the wildcard connection atoms onto the compound. It selects the head/tail hydrogen pair whose bond vectors are most antiparallel (straightest chain) and assembles N units with `mbuild.lib.recipes.polymer.Polymer`. Polyester ends are capped as carboxylic acids. Building from SMILES avoids a PDB round-trip that can stretch backbone bonds and break downstream atom typing.

C. Geometry optimisation (OpenBabel)

Both the monomer and the assembled chain are minimised with an OpenBabel force field (default MMFF94, conjugate gradients), with an automatic MMFF94 → UFF → Ghemical fallback so exotic monomers still relax.

D. Force-field assignment & topology

Two typing routes are supported. Moltemplate-native force fields are written as `.lt` files and compiled by `moltemplate.sh` into LAMMPS data/settings. DL_FIELD force fields are typed by the `dl_field` program using DL_F Notation, which auto-assigns atom types against its library and emits LAMMPS or GROMACS topology directly. For OPLS-AA specifically, PAAF's SMARTS-based typer emits valid numeric OPLS atom-type IDs; a manual typing dialog is available for overrides.

E. Box packing

For LAMMPS, PAAF replicates the single-chain data file into an N-chain system using `packmol` (with a random seed for varied layouts) and non-hybrid type-ID offsetting; a numpy grid packer is the fallback if `packmol` is unavailable. For GROMACS, PAAF uses `gm insert-molecules` — optionally pre-minimising the single chain, then inserting either an explicit count or a target-atom-derived count of copies, and updating the topology's molecule count. A pure-Python random-rotation packer is the fallback when `gm` is not installed.

F. Blends

Multi-component systems merge several single-chain data files with per-component cumulative type-ID offsets across atom, bond, angle, dihedral and improper types; all coefficient sections are concatenated with the same offsets; and `packmol` packs every copy of every species into a shared box. A merged LAMMPS input file carries the offset `pair_coeff` commands.

G. Software stack & reproducibility

GUI: PyQt5. Chemistry: OpenBabel, RDKit. Chain assembly: mBuild, NetworkX. Typing / topology: moltemplate.sh, DL_FIELD 4.13. Packing: packmol, GROMACS (`gm insert-molecules`). MD engines: LAMMPS, GROMACS. Every GUI run serialises to the same Config object the headless pipeline uses, so a GUI-built system and a scripted one are identical; random seeds (copolymer sequence, packmol layout) are configurable for exact reproduction.

GUI	PyQt5
Chemistry	OpenBabel · RDKit
Chain assembly	mBuild · NetworkX
Typing / topology	moltemplate.sh · DL_FIELD 4.13
Packing	packmol · GROMACS (gmx insert-molecules)
MD engines	LAMMPS · GROMACS